

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«ОМСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»  
Операционные системы

**Отчёт**  
по лабораторной работе №6

Выполнил:

Студент группы ПИ-221

Нурсалимов Данияр Ерланович

Проверил(а): \_\_\_\_\_

\_\_\_\_\_

Дата сдачи отчета: \_\_\_\_\_

---

**Задание.** В работе нужно разработать пять программ – одну для процесса родителя, две – для дочерних процессов, еще две – для «внучатых» процессов. Все программы осуществляют вывод сообщений о своем «родственном» положении (например, «родитель», «дочерний», «внук» и номер шага в цикле вывода таких сообщений, между которыми следует задавать задержки в пределах 1-2 секунд. Число повторений таких выводов сообщений должно составлять 10 – 20 для каждой программы, задержки для них следует выбирать различные, но незначительно – отличающиеся на 30–70%). На 7 шаге для вывода сообщений родительская программа отдает приказ об уничтожении первой дочерней и соответствующее дополнительное сообщение об этом на экран, на 11 шаге родительская программа выдает приказ об уничтожении второй дочерней и ее потомка. Все такие программы, создающие новый процесс, должны передавать в дочерний процесс текстовую информацию о присваиваемом ей текстовом имени дочернего (аналогичном «человеческим» именам). Это имя имеет исключительно иллюстративное значение и должно выводиться каждым процессом кроме исходного родительского с соответствующим примечанием о его назначении. (Базовый вариант.)

**Исходный код базовой программы “parent.c” на Windows:**

```
#include <windows.h>
#include <stdio.h>

int main()
{
    int k;
    DWORD rc1,rc2;
    STARTUPINFO si1,si2;
    PROCESS_INFORMATION pi1,pi2;
    HANDLE hjob = CreateJobObject(NULL, NULL);//Создание задачи

    printf("Demonstration processes, Main Proccess\n");
```

```
memset(&si1, 0, sizeof(STARTUPINFO)); si1.cb=sizeof(si1);
memset(&si2, 0, sizeof(STARTUPINFO)); si2.cb = sizeof(si2);
```

```
rc1=CreateProcess("child1.exe", "ch1", NULL, NULL, TRUE, //Создание
процессов
```

```
    NORMAL_PRIORITY_CLASS, NULL, NULL, &si1, &pi1);
```

```
rc2 = CreateProcess("child2.exe", "ch2", NULL, NULL, TRUE,
```

```
    NORMAL_PRIORITY_CLASS, NULL, NULL, &si2, &pi2);
```

```
if ((!rc1)||(!rc2))//Проверка успешности создания процессов
```

```
{
```

```
    printf("Error create Process, codeError = %ld\n", GetLastError());
```

```
    getchar(); return 0;
```

```
}
```

```
AssignProcessToJobObject(hjob, pi2.hProcess); //Создание связи м-ду
задачей и 2 процессом
```

```
for (k=0; k<10; k++)
```

```
{
```

```
printf("I am Parent... (my K=%d)\n", k);
```

```
if (k == 7)//Уничтожение 1-го процесса
```

```
{
```

```
    printf("I will delete Child-1\n");
```

```
    TerminateProcess(pi1.hProcess, 0);
```

```
    printf("Child-1 has been deleted\n");
```

```
}
```

```
if (k == 9)//Уничтожения 2-го процесса и его потомка
```

```
{
```

```
    printf("I will delete Child-2 and Child-2.1\n");
```

```
    TerminateJobObject(hjob, 0);
```

```

        printf("Child-2 and Child-2.1 has been deleted\n");
    }
    Sleep(3000);
}
CloseHandle(pi1.hProcess); CloseHandle(pi1.hThread);
CloseHandle(pi2.hProcess); CloseHandle(pi2.hThread);
return 0;
}

```

### **Исходный код базовой программы “child1.c” на Windows:**

```

// Демонстрация порождения процессов. Child Process.
#include <stdio.h>
#include <windows.h>
int main(int argc, char**argv)
{
    int k;
    DWORD rc;
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    memset(&si, 0, si.cb = sizeof(si));
    char* childName = "ch1";
    char* buf = (char*)malloc(strlen(argv[0]) + strlen(childName) +
1); //Заполнение буфера

    sprintf(buf, "%s %s", argv[0], childName); // Вывод буфера обмена
    rc = CreateProcess("child1-1.exe", buf, NULL, NULL, TRUE,
BELOW_NORMAL_PRIORITY_CLASS, NULL, NULL, &si, &pi); //Создание
процесса
    if (!rc) //Проверка успешности создания процесса
    {

```

```

    printf("Error create Process, codeError = %ld\n", GetLastError());
    getchar(); return 0;
}

```

```

printf("Demonstration processes, Child-1 Proccess\n");
for (k=0; k<20; k++)
{printf("I am Child-1 %s... (k=%d)\n\r",argv[0], k); Sleep(3000); }
CloseHandle(pi.hProcess); CloseHandle(pi.hThread);
return 0;
}

```

### **Исходный код базовой программы “child2.c” на Windows:**

// Демонстрация порождения процессов. Child Process.

```
#include <stdio.h>
```

```
#include <windows.h>
```

```
int main(int argc,char** argv)
```

```
{
```

```
int k;
```

```
DWORD rc;
```

```
STARTUPINFO si;
```

```
PROCESS_INFORMATION pi;
```

```
memset(&si, 0, si.cb = sizeof(si));
```

```
char* childName = "ch2";
```

```
char* buf = (char*)malloc(strlen(argv[0]) + strlen(childName) +
1);//Заполнение буфера
```

```
sprintf(buf, "%s %s", argv[0], childName); // Вывод буфера обмена
```

```
rc = CreateProcess("child2-1.exe", buf, NULL, NULL, TRUE,
BELOW_NORMAL_PRIORITY_CLASS, NULL, NULL, &si, &pi); //Создание
процесса
```

```
if (!rc) //Проверка успешности создания
```

### **Исходный код базовой программы “child1-1.c” на Windows:**

```
// Демонстрация порождения процессов. Child Process.
```

```
#include <stdio.h>
```

```
#include <windows.h>
```

```
int main(int argc, char** argv)
```

```
{
```

```
int k;
```

```
printf("Demonstration processes, Child-1.1 Proccess\n");
```

```
for (k = 0; k < 30; k++)
```

```
{
```

```
printf("I am Child-1.1 %s, my parent %s... (k=%d)\n\r",
```

```
argv[1],argv[0],k); Sleep(3000);
```

```
}
```

```
return 0;
```

```
}
```

### **Исходный код базовой программы “child2-1.c” на Windows:**

```
// Демонстрация порождения процессов. Child Process.
```

```
#include <stdio.h>
```

```
#include <windows.h>
```

```
int main(int argc, char** argv)
```

```
{
```

```
int k;
```

```
printf("Demonstration processes, Child-2.1 Proccess\n");
```

```
for (k = 0; k < 30; k++)
```

```
{  
    printf("I am Child-2.1 %s, my parent %s... (k=%d)\n\r", argv[1],  
argv[0], k); Sleep(3000);  
}  
return 0;  
}
```

#### **Результаты базового задания:**

```
I will delete Child-1  
Child-1 has been deleted  
I am Child-1.1 ch1, my parent ch1... (k=7)
```

```
I will delete Child-2 and Child-2.1  
Child-2 and Child-2.1 has been deleted  
I am Child-1.1 ch1, my parent ch1... (k=9)  
I am Child-1.1 ch1, my parent ch1... (k=10)
```